

Code Library · Stage 4

Six tiny programs about MOVING things around the screen. Copy, paste, and tweak.

First, a tiny bit of background

Every "thing" in a pygame game is really a **Rect** - a rectangle with four numbers: **x** (left edge), **y** (top edge), **width**, and **height**. Move it by changing x or y. Redraw it every frame. That's all animation is.

To read the keyboard each frame, ask pygame which keys are being held:

```
# the heart of Stage 4
player = pygame.Rect(370, 520, 60, 40)    # x, y, width, height
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]: player.x -= 5
if keys[pygame.K_RIGHT]: player.x += 5
```

Common key names: K_LEFT, K_RIGHT, K_UP, K_DOWN, K_SPACE, K_a, K_d, K_w, K_s. You can mix and match.

How to use this handout

Each game has three parts: **what it does**, **the code**, and **challenges**. Copy any code block into the Trinket on the lesson page, then press the green **Run** button. Change a number, tweak a key - see what happens.

Game 1 · The Player (still)

What it does

Draws a green rectangle near the bottom-centre of the screen. It doesn't move yet - this is the starting point we'll add movement to in Game 2.

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("The Player (still)")

GREEN = (0, 220, 100)
SPACE = (0, 0, 50)

# the player is just a Rect: (x, y, width, height)
player = pygame.Rect(370, 520, 60, 40)

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill(SPACE)
    pygame.draw.rect(screen, GREEN, player)
    pygame.display.flip()

pygame.quit()
```

Try these one by one

Player 1 Change the player to yellow.

```
YELLOW = (255, 215, 0)
pygame.draw.rect(screen, YELLOW, player)
```

Player 2 Make it bigger - change width and height to 120 and 80.

```
player = pygame.Rect(370, 520, 120, 80)
```

Player 3 Put it at the top of the screen by changing y to 20.

```
player = pygame.Rect(370, 20, 60, 40)
```

Game 2 · Move the Player

What it does

Game 1 + keyboard. Hold LEFT or RIGHT and the player slides. Notice it can wander off the edge - we'll fix that in Game 3.

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Move the Player - press LEFT / RIGHT")

GREEN = (0, 220, 100)
SPACE = (0, 0, 50)

player = pygame.Rect(370, 520, 60, 40)
player_speed = 5

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # ask which keys are held down
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= player_speed
    if keys[pygame.K_RIGHT]:
        player.x += player_speed

    screen.fill(SPACE)
    pygame.draw.rect(screen, GREEN, player)
    pygame.display.flip()

pygame.quit()
```

Try these one by one

Move 1 Make the player 3x faster.

```
player_speed = 15
```

Move 2 Make it crawl slowly.

```
player_speed = 1
```

Move 3 Swap the controls: LEFT goes right and RIGHT goes left.

```
if keys[pygame.K_LEFT]: player.x += player_speed
if keys[pygame.K_RIGHT]: player.x -= player_speed
```

Game 3 · Stay on Screen

What it does

Same as Game 2 but with boundary checks. The player stops at the edges instead of escaping. This is the version we'll use in Galaxy Defender.

```
import pygame

pygame.init()
WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Stay on Screen")

GREEN = (0, 220, 100)
SPACE = (0, 0, 50)

player = pygame.Rect(370, 520, 60, 40)
player_speed = 5

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] and player.x > 0:
        player.x -= player_speed
    if keys[pygame.K_RIGHT] and player.x < WIDTH - player.width:
        player.x += player_speed

    screen.fill(SPACE)
    pygame.draw.rect(screen, GREEN, player)
    pygame.display.flip()

pygame.quit()
```

Try these one by one

Stay 1 Make the screen wider (1200) and watch the player roam further.

```
WIDTH, HEIGHT = 1200, 600
```

Stay 2 Leave a 50-pixel gap on the right by changing `WIDTH - player.width` to `WIDTH - player.width - 50`.

Stay 3 Add a check so the player can never go past `x=200` on the LEFT (a wall).

```
if keys[pygame.K_LEFT] and player.x > 200: player.x -= player_speed
```

Game 4 · Up / Down / Left / Right

What it does

All four arrow keys. Move freely around the whole screen, staying inside the edges. Useful for top-down games like Pac-Man.

```
import pygame

pygame.init()
WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Up / Down / Left / Right")

YELLOW = (255, 215, 0)
SPACE = (0, 0, 50)

player = pygame.Rect(370, 280, 60, 40)
speed = 5

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] and player.x > 0: player.x -= speed
    if keys[pygame.K_RIGHT] and player.x < WIDTH - player.width: player.x += speed
    if keys[pygame.K_UP] and player.y > 0: player.y -= speed
    if keys[pygame.K_DOWN] and player.y < HEIGHT - player.height: player.y += speed

    screen.fill(SPACE)
    pygame.draw.rect(screen, YELLOW, player)
    pygame.display.flip()

pygame.quit()
```

Try these one by one

Updn 1 Use WASD instead of arrows: replace K_LEFT/K_RIGHT/K_UP/K_DOWN with K_a/K_d/K_w/K_s.

Updn 2 Make the player a circle instead of a rect by replacing draw.rect with draw.circle. (You'll need to use player.center.)

```
pygame.draw.circle(screen, YELLOW, player.center, 25)
```

Updn 3 Make UP and DOWN twice as fast as LEFT and RIGHT.

```
if keys[pygame.K_UP] and player.y > 0: player.y -= speed * 2
```

Game 5 · Two Players

What it does

Two players, one keyboard. Red uses A/D, blue uses LEFT/RIGHT. The first multiplayer game you've ever built!

```
import pygame

pygame.init()
WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Two Players - A/D and LEFT/RIGHT")

RED = (230, 80, 80)
BLUE = (80, 130, 230)
SPACE = (0, 0, 50)

p1 = pygame.Rect(200, 520, 60, 40) # red, controlled by A / D
p2 = pygame.Rect(540, 520, 60, 40) # blue, controlled by arrows
speed = 5

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()

    if keys[pygame.K_a] and p1.x > 0: p1.x -= speed
    if keys[pygame.K_d] and p1.x < WIDTH - p1.width: p1.x += speed

    if keys[pygame.K_LEFT] and p2.x > 0: p2.x -= speed
    if keys[pygame.K_RIGHT] and p2.x < WIDTH - p2.width: p2.x += speed

    screen.fill(SPACE)
    pygame.draw.rect(screen, RED, p1)
    pygame.draw.rect(screen, BLUE, p2)
    pygame.display.flip()

pygame.quit()
```

Try these one by one

Two 1 Give player 1 W and S for up/down too.

```
if keys[pygame.K_w] and p1.y > 0: p1.y -= speed
```

Two 2 Make player 2 twice as wide.

```
p2 = pygame.Rect(540, 520, 120, 40)
```

Two 3 Add a third player (PURPLE) controlled by J and L.

```
p3 = pygame.Rect(400, 460, 60, 40)
# then if keys[pygame.K_j] / K_l: p3.x +/- speed
```

Game 6 · Auto-Bouncer (no keyboard)

What it does

A square that moves all by itself, bouncing between the left and right edges. No keys needed. This is exactly how the enemies in Lesson 6 will move.

```
import pygame

pygame.init()
WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Auto-Bouncer (no keys needed)")
clock = pygame.time.Clock()

YELLOW = (255, 215, 0)
SPACE = (0, 0, 50)

square = pygame.Rect(0, 280, 60, 40)
dx = 4 # how many pixels it moves each frame

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    square.x += dx
    # bounce off the edges
    if square.x <= 0 or square.x >= WIDTH - square.width:
        dx = -dx

    screen.fill(SPACE)
    pygame.draw.rect(screen, YELLOW, square)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Try these one by one

- Bounce 1** Make it bounce VERTICALLY instead. Use square.y and HEIGHT, change dx to dy.
- Bounce 2** Make it faster: change dx = 4 to dx = 10.
- Bounce 3** Add a second square that bounces in the opposite direction at the same speed.

Where this takes us next

You now control a player. In Lesson 5 we'll add **bullets** - things that appear when you press SPACE and shoot upwards. The trick? Bullets are just little rectangles whose y goes down a tiny bit every frame. Exactly the Auto-Bouncer pattern.

Challenge: combine Game 4 (4-way movement) with Game 1's drawing trick - draw a smiley face that follows the player around. Screenshot and share!